

BM25 Lexical Search

Video Link: [BM25 Lexical Search](#)

When building RAG pipelines, you'll quickly discover that semantic search alone doesn't always return the best results. Sometimes you need exact term matches that semantic search might miss. The solution is to combine semantic search with lexical search using a technique called BM25.

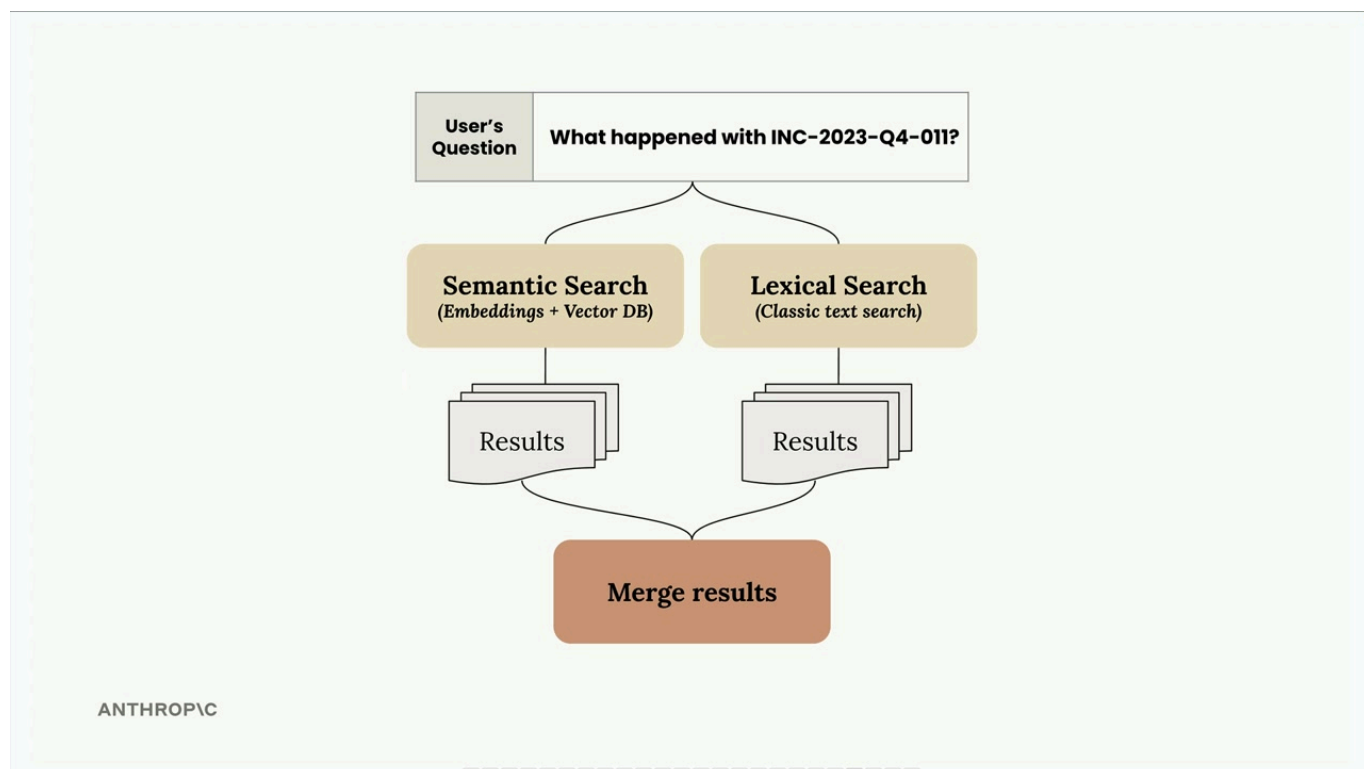
The Problem with Semantic Search Alone

Let's say you're searching for a specific incident ID like "INC-2023-Q4-011" in a document. While semantic search excels at understanding context and meaning, it might return sections that are semantically related but don't actually contain the exact term you're looking for.

In the example shown in the video, semantic search returned the cybersecurity section (which does contain the incident ID) but also returned a financial analysis section that doesn't mention the incident at all. This happens because semantic search focuses on conceptual similarity rather than exact term matching.

Hybrid Search Strategy

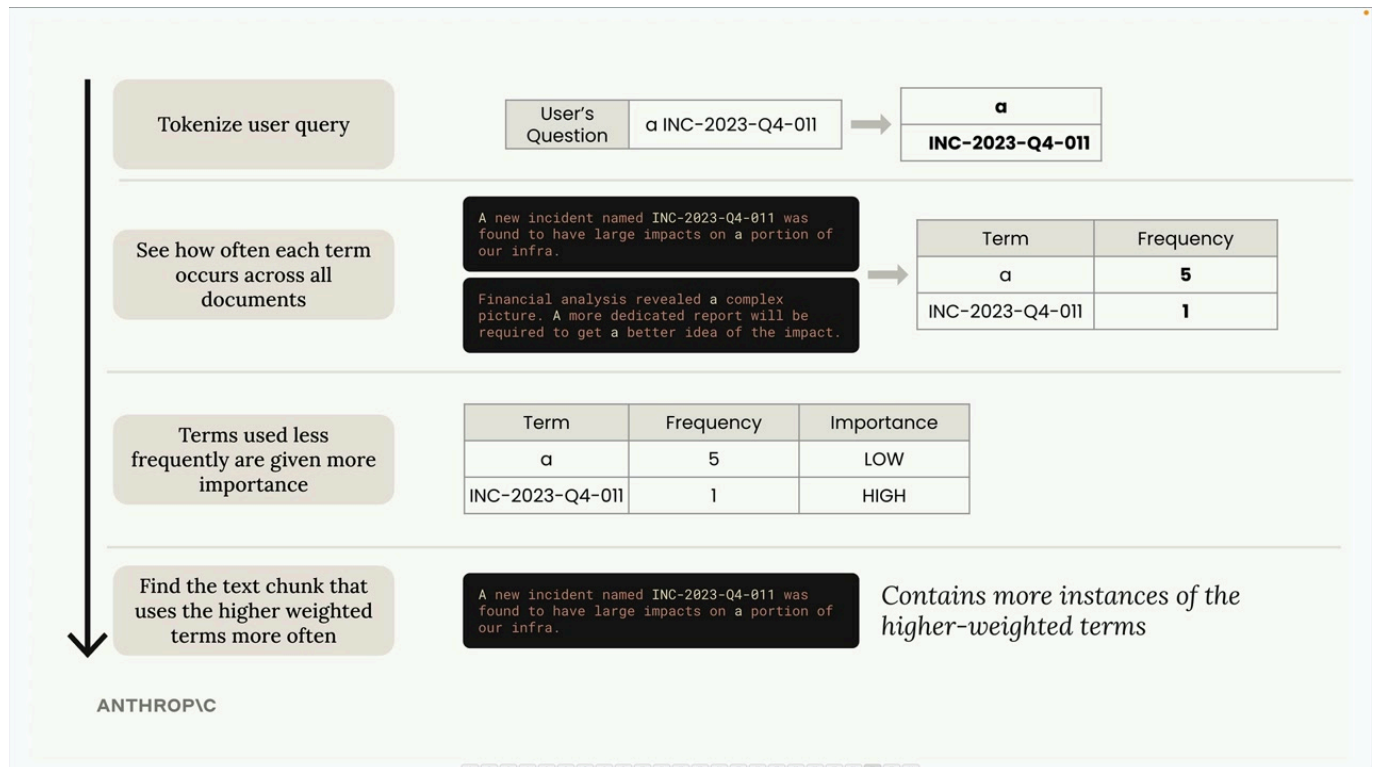
The solution is to run both semantic and lexical searches in parallel, then merge the results. This gives you the best of both worlds:



- **Semantic search** finds conceptually related content using embeddings
- **Lexical search** finds exact term matches using classic text search
- **Merged results** combine both approaches for better accuracy

How BM25 Works

BM25 (Best Match 25) is a popular algorithm for lexical search in RAG systems. Here's how it processes a search query:



Step 1: Tokenize the query

Break the user's question into individual terms. For example, "a INC-2023-Q4-011" becomes ["a", "INC-2023-Q4-011"].

Step 2: Count term frequency

See how often each term appears across all your documents. Common words like "a" might appear 5 times, while specific terms like "INC-2023-Q4-011" might appear only once.

Step 3: Weight terms by importance

Terms that appear less frequently get higher importance scores. The word "a" gets low importance because it's common, while "INC-2023-Q4-011" gets high importance because it's rare.

Step 4: Find best matches

Return documents that contain more instances of the higher-weighted terms.

Implementing BM25 Search

Here's how to set up a basic BM25 search system:

```
# 1. Chunk your text by sections
chunks = chunk_by_section(text)

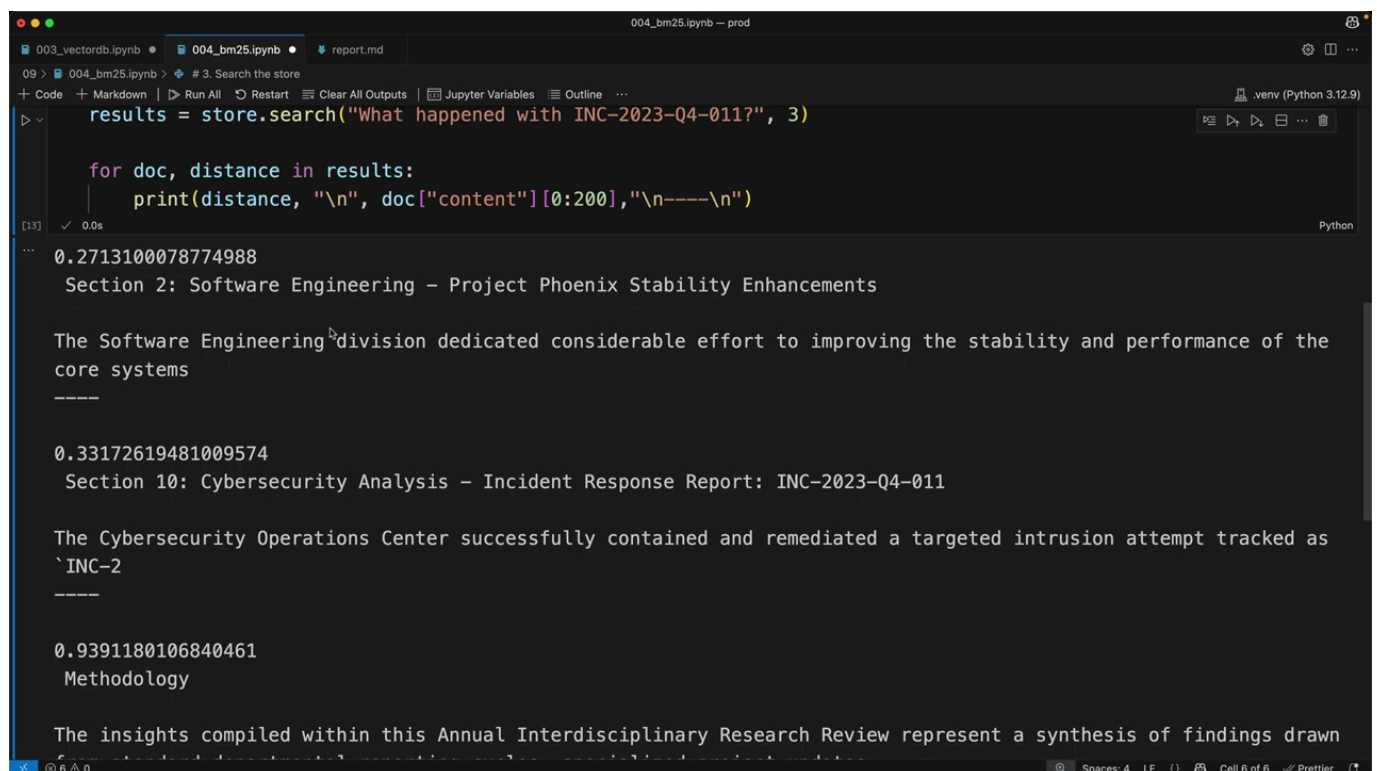
# 2. Create a BM25 store and add documents
```

```
store = BM25Index()
for chunk in chunks:
    store.add_document({"content": chunk})

# 3. Search the store
results = store.search("What happened with INC-2023-Q4-011?", 3)

# Print results
for doc, distance in results:
    print(distance, "\n", doc["content"][:200], "\n----\n")
```

When you run this search, you'll get much better results than semantic search alone. The BM25 algorithm prioritizes sections that actually contain your specific search terms, especially rare terms like incident IDs.



```
0.271310078774988
Section 2: Software Engineering - Project Phoenix Stability Enhancements

The Software Engineering division dedicated considerable effort to improving the stability and performance of the
core systems
-----

0.33172619481009574
Section 10: Cybersecurity Analysis - Incident Response Report: INC-2023-Q4-011

The Cybersecurity Operations Center successfully contained and remediated a targeted intrusion attempt tracked as
`INC-2
-----

0.9391180106840461
Methodology

The insights compiled within this Annual Interdisciplinary Research Review represent a synthesis of findings drawn
```

Notice how the results now properly prioritize the Software Engineering section and Cybersecurity section - both of which actually contain the incident ID you're searching for.

Why This Works Better

BM25 excels at finding exact matches because it:

- Gives higher weight to rare, specific terms
- Ignores common words that don't add search value
- Focuses on term frequency rather than semantic meaning
- Works especially well for technical terms, IDs, and specific phrases

The key insight is that both search methods have complementary strengths. Semantic search understands context and meaning, while lexical search ensures you don't miss exact term matches. By combining them, you create a more robust search system that handles both conceptual queries and specific lookups effectively.

In the next step, you'll learn how to merge results from both search systems to create a unified hybrid search experience.